Chapter 1

Probability in Reasoning, Inference, and Learning

1.1 Probability as the language of uncertainty

How do we represent information and uncertainty mathematically? When you type "good" into your phone, the keyboard offers a few likely continuations: perhaps "morning", "luck", and "night". These suggestions reflect a state of knowledge: "morning" may follow with probability 0.45, "luck" with probability 0.25, "night" with probability 0.15, and other words with the remaining 0.15 (the numbers are illustrative). A probability distribution over the possible next words captures both what the options are and how strongly each is expected. It also encodes uncertainty: your phone does not know for sure what you are going to write next.

Probability theory is our main tool for representing knowledge and uncertainty. Additionally, it provides a framework for formulating assumptions. In this chapter, we will study the role of probability in reasoning, inference, and machine learning. We first describe these tasks, bearing in mind that there is no universally accepted definition for them.

Probabilistic reasoning: Given probability distributions for a set of random variables, we can use probabilistic reasoning to draw conclusions about unobserved variables based on observed variables.

Inference: Drawing conclusions about the real world or a given system based on evidence or observations. Our models typically aim to reflect reality. For example, determining whether a gene is linked to a disease.

(Supervised) machine learning: Learning from examples of (input, output) pairs and generalizing to predict the output for new inputs. For example, predicting the price of a house based on its features. Machine learning models and algorithms are not necessarily focused on representing reality and are more concerned with the accuracy of their predictions.

When considering these problems, we deal with uncertainty. Sources of uncertainty, for example in machine learning, include:

- **Noise**: aggregate contribution of factors that we do not (wish to) consider (models focus on the most important quantities). For example, your height can be predicted well from your parents' heights and general patterns of nutrition and sleep, which we can include in our model. But it may also depend on every meal you ate and how much you slept each night throughout your youth, which we cannot include.
- Finite sample size: finite size of data makes it impossible to fully determine relationships (i.e., probability distributions) as some configuration may never happen or happen a few times in finite data.

1.1.1 Relationships, distributions, and probabilistic reasoning

Is there any relationship between the arrival times of two people working at a business (opening at 9:00 am), both living in the same area? If so, how can we represent this relationship? How can we make predictions about one being late given the other is late (e.g., if we need at least one person to be present)?

In the same way that we can encode our information about a random quantity as a distribution, we can encode information about random quantities, as well as their relationships, as joint distributions.

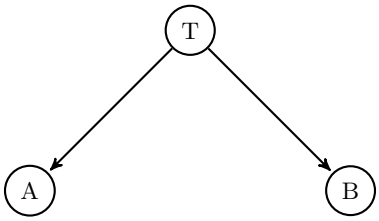
In our example, there's obviously a relationship, that is, the arrival times are not independent. For example, both are affected by traffic. Let

$$\begin{aligned} T_0 &: \text{normal traffic} \\ T_1 &: \text{heavy traffic} \\ A_0 &: \text{Alice is on time} \\ A_1 &: \text{Alice is late} \\ B_0 &: \text{Bob is on time} \\ B_1 &: \text{Bob is late} \end{aligned}$$

and assume

$$\begin{aligned} \Pr(T_0) &= 0.65, \\ \Pr(A_0|T_0) &= 0.9, \\ \Pr(B_0|T_0) &= 0.82, \\ \Pr(A_0|T_1) &= 0.5, \\ \Pr(B_0|T_1) &= 0.15. \end{aligned}$$

Finally, conditioned on the traffic situation, Alice and Bob's arrival times are independent. This information completely determines all probabilities. As we will see in greater depth later, the fact that Alice and Bob's arrival times are only related through traffic can be shown *graphically* as



Causal reasoning:

$$\begin{aligned} \Pr(A_0) &= \Pr(T_0)\Pr(A_0|T_0) + \Pr(T_1)\Pr(A_0|T_1) \\ &= (0.65 \times 0.9) + (0.35 \times 0.5) = 0.76, \\ \Pr(B_0) &= \Pr(T_0)\Pr(B_0|T_0) + \Pr(T_1)\Pr(B_0|T_1) \\ &= (0.65 \times 0.82) + (0.35 \times 0.15) = 0.5855. \end{aligned}$$

Evidential reasoning (inverse probabilities, uses Bayes rule):

$$\begin{aligned} \Pr(T_0|A_0) &= \Pr(A_0|T_0)\Pr(T_0)/\Pr(A_0) = 0.65 \times 0.9/0.76 = 0.7697 \\ \Pr(T_0|B_0) &= \Pr(B_0|T_0)\Pr(T_0)/\Pr(B_0) = 0.65 \times 0.82/0.5855 = 0.9103 \end{aligned}$$

The common cause makes the events A_i and B_i dependent. Recall that two events E_1 and E_2 are independent,

denoted $E_1 \perp\!\!\!\perp E_2$ if $\Pr(E_1E_2) = \Pr(E_1)\Pr(E_2)$, or, if $\Pr(E_2) \neq 0$, $\Pr(E_1|E_2) = \Pr(E_1)$. We have

$$\begin{aligned}
\Pr(A_0|B_0) &= \Pr(A_0B_0)/\Pr(B_0) \\
\Pr(A_0B_0) &= (0.65 \times 0.82 \times 0.9) + (0.35 \times 0.15 \times 0.5) = 0.506 \\
\Pr(A_0|B_0) &= 0.506/0.5855 \approx 0.864 \neq \Pr(A_0) \\
\Pr(B_0|A_0) &= 0.506/0.76 = 0.6658 \neq \Pr(B_0)
\end{aligned}$$

So $A_0 \not\perp\!\!\!\perp B_0$.

However, they are *conditionally independent*, by assumption

$$\Pr(A_0B_0|T_0) = \Pr(A_0|T_0)\Pr(B_0|T_0),$$

which is denoted as $A_0 \perp\!\!\!\perp B_0|T_0$.

What is the source of uncertainty in this problem? Since we have assumed the distribution is known, sample size is not an issue. The source is noise. For example, if we had information about other factors affecting Bob, e.g., how reliable his car is, if he needs to drop off his kids, etc., we could reduce the amount of noise and make better predictions.

1.2 Inference and machine learning

The reasoning in the previous section relied on distributions that were fully known: five numbers specified the model, and probability answered every question we asked. The other two problem types, inference and machine learning, begin instead from data, because the distributions they need are unknown. Each is illustrated here briefly, together with its sources of uncertainty. The full treatments come in later chapters.

1.2.1 Inference

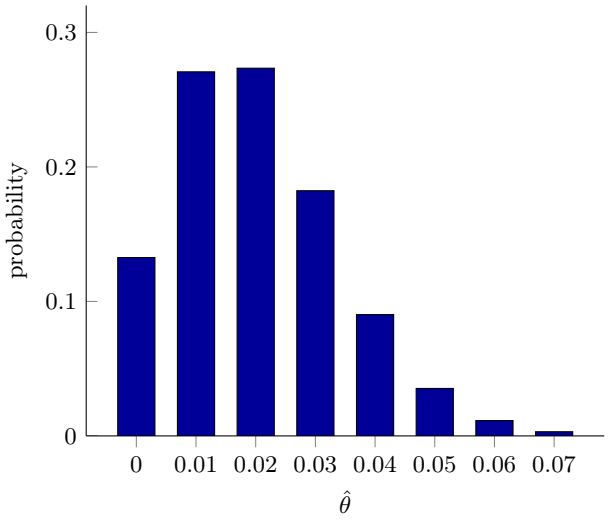
Inference is the process of using data to draw conclusions about the real world; for example, estimating an unknown quantity. Suppose that the probability that someone with a given allele of a gene will develop a certain disease is θ. We are interested in determining θ; an answer is a number for θ together with a statement of how sure we are about it. To find one, we collect data: among a sample of 100 people with this allele, 2 have the disease. How we approach this problem depends on whether we view θ as deterministic or random: the same data and the same question, but two different notions of what "unknown" means.

1.2.1.1 The frequentist view

In the **frequentist view**, θ is fixed and deterministic, but unknown. The frequentist framework does not prescribe a single way to estimate θ from data; many **estimators** are possible. Suppose we take the **sample mean** $\hat{\theta} = 2/100 = 0.02$ as our estimate. How accurate is this estimate? **We cannot know: judging it would require knowing the true value of θ.** What we can ask is how accurate the *estimator* is, and that depends on two things:

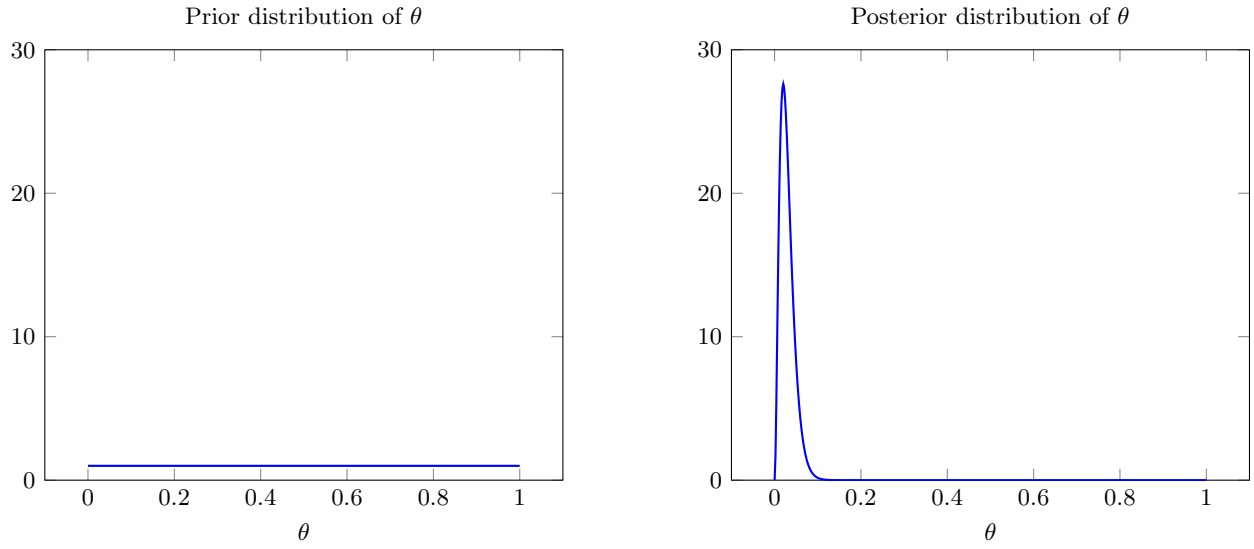
- how well a random sample represents the true value, and
- how well the method (here, the sample mean) extracts information from the sample.

Accuracy in the frequentist view is instead a property of the procedure, evaluated over all the samples we could have gotten. If θ were really 0.02, the number of diseased people in a sample of 100 would have the distribution $\text{Bin}(100, 0.02)$ and $\hat{\theta}$ would be that number divided by 100; the resulting distribution of $\hat{\theta}$ is shown below. The estimate equals 0.02 only about 27% of the time. Quantifying this spread, and resolving the apparent contradiction of assuming we know θ in order to study the estimator, are the subjects of the chapter on frequentist parameter estimation.



1.2.1.2 The Bayesian view

In the **Bayesian view**, θ is a random variable: we could be living in a universe where θ is 0.01, or 0.05, or any other value, and in many cases we will never find out which. Treating θ as a fixed number would then be a contradiction. We describe our initial belief about θ using a **prior distribution**, say a uniform distribution over $[0, 1]$, and update it using the two observed cases to obtain the **posterior distribution** $p(\theta \mid \text{data})$. How to calculate the posterior from the prior and the data is the subject of the chapter on Bayesian parameter estimation; for our data it is shown below. The posterior is peaked at 0.02, its mode, with mean 0.029, but still allows other values. It is the whole answer: a number for θ and how sure we are about it, in one distribution.



1.2.1.3 Source of uncertainty

In both views, the source of uncertainty is **finite sample size**. If we knew the disease status of 20 out of 1000 people with the allele, the same rate with ten times the data, both the distribution of the estimate $\hat{\theta}$ and the posterior distribution of θ would be more concentrated, with lower variance. Estimating θ from the sample and quantifying the remaining uncertainty are the subjects of the chapters on frequentist and Bayesian parameter estimation.

1.2.2 Supervised machine learning

Consider the house-price problem from the start of the chapter: predict the sale price of a house from features such as location, lot size, square footage, and number of bedrooms. More generally, supervised machine learning problems have the following components:

- **Data**: $\mathcal{D} = \{(x_1, y_1), ..., (x_N, y_N)\}, x_i \in \mathcal{X}, y_i \in \mathcal{Y}$. $\mathcal{X}$ is called the feature space, and $\mathcal{Y}$ is called the label space. As an example, each x_i could be a vector providing information about a house, e.g., (location, lot size, square footage, number of bedrooms, ...), and y_i can be the sale price of the house.
- **Goal**: Find the "best" function f to predict y corresponding to a given x. In other words, the function f produces an estimate $\hat{y} = f(x)$ of y given data x. Continuing our example, y would be the true but unknown price of the house with features x, and $f(x)$ would be a prediction (similar to what Zillow does).
- **Evaluation**: How do we define "*best*"? For a given data point (x, y), evaluate the success of f using a loss function $L(y, f(x))$, e.g., $L(y, f(x)) = |y - f(x)|$. Ideally, we would like to minimize the expected loss over all possible outcomes weighted by their probabilities, so we define

$$\mathcal{L}(f) = \mathbb{E}[L(Y, f(X))], \tag{1.1}$$

  also known as the **population risk**, where the expectation is over the distribution $p(x, y)$ of (X, Y).
- **Learning Algorithm**: The algorithm that tries to find

$$f^* = \arg\min_f \mathcal{L}(f) = \arg\min_f \mathbb{E}[L(Y, f(X))]. \tag{1.2}$$

If we have the joint distribution $p(x, y)$, we can solve this problem exactly, at least in principle (see Section 1.2.2.2). But what we actually have is the data set $\mathcal{D}$. We will discuss how to address this mismatch through empirical risk minimization or by estimating the unknown distribution $p(x, y)$ from $\mathcal{D}$ and then predicting with the estimate.

1.2.2.1 Source of uncertainty

Both sources of uncertainty appear in the house-price problem. The features do not determine the price: two houses identical on paper can sell for different amounts. There is **noise** from factors left out of the model. The relationship between features and prices must also be learned from a finite record of past sales, so **finite sample size** introduces uncertainty about that relationship. Section 1.3 makes this limitation concrete by showing how quickly finite data runs out when a language model is learned by counting.

1.2.2.2 What would knowing the distribution give us?

If the joint distribution $p(x, y)$ of the features X and the target Y were known exactly, the supervised-learning problem would be solved: the expected loss could be evaluated, and the optimal predictor f^* could be found without learning from a finite dataset. Two common problems in supervised learning illustrate this:

- **Regression**: $\mathcal{Y}$ consists of **scalars or vectors of reals**. For example, predicting stock price based on financial information, or determining the score someone will assign a movie based on previous scores. A common loss function is the **quadratic** or **squared error** loss function:

$$L(y, f(x)) = (y - f(x))^2. \tag{1.3}$$

  It can be shown that for this loss, *if the distribution is known*,

$$f^*(x) = \mathbb{E}[Y|X = x]. \tag{1.4}$$

- **Classification**: $\mathcal{Y}$ consists of **classes or categories**. For example, speech recognition, handwriting recognition, the presence or absence of a disease. A common loss function is the **0-1 loss**:

$$L(y, f(x)) = \begin{cases} 1, & \text{if } y \neq f(x). \\ 0, & \text{if } y = f(x). \end{cases} \tag{1.5}$$

  In this case, *if the distribution is known*, then the best classifier is

$$f^*(x) = \arg\max_{y \in \mathcal{Y}} p(y|x). \tag{1.6}$$

We emphasize again that to solve the problem optimally as in (1.4) and (1.6), we need to know the joint distribution of (X, Y) or the conditional distribution of Y given X.

Knowing the joint buys one more thing: **generation**. We can sample a fresh pair (X, Y) from $p(x, y)$, or, given x, sample Y from $p(y|x)$, using the sampling methods of Chapter 0. In the context of **large language models**, discussed later, the task may be to generate a response y given a prompt x.

1.3 From Coin Flips to Language Models

A Bernoulli random variable is about as simple as a model gets: one number, the probability of heads, says everything there is to say about a biased coin. A month of daily weather asks for more, because a model of the whole record has to say how each day depends on the days before it. A model of English asks for much more again, since it must give a probability to every sentence anyone might write, and with a vocabulary of ten thousand words there are 10^{80} strings of twenty words to account for. All three are joint distributions over the quantities we care about. What separates them is how complex they are and how much structure we impose to keep the model small enough to learn from data.

The tools for imposing that structure are already in Chapter 0: the chain rule, which factors a joint distribution into conditional distributions, and conditional independence, which lets each factor forget most of what it conditions on. This section applies them to sequences and follows one construction the whole way, from a sequence of Bernoulli variables recording rain or no rain to the next-token models that large language models are built from.

1.3.1 Why the full joint distribution grows so quickly

Suppose we record the weather at one location for d days. Let $X_t = 1$ mean that it rains on day t, and let $X_t = 0$ mean that it does not. Our goal is to model the complete sequence

$$X_{1:d} = (X_1, \ldots, X_d),$$

not just the weather on one particular day. With such a model, we could assign a probability to an event such as a week of continuous rain, or we could generate possible weather sequences.

For three days, there are eight possible sequences:

$$000,\ 001,\ 010,\ 011,\ 100,\ 101,\ 110,\ 111.$$

In general, d binary observations have 2^d possible sequences. A table that gives a probability to every sequence therefore needs

$$2^d - 1 \tag{1.7}$$

free parameters. The subtraction by one appears because all the probabilities must sum to one. For a 30-day record, the number is

$$2^{30} - 1 = 1{,}073{,}741{,}823.$$

We would have trouble even filling such a table with useful estimates: an ordinary weather dataset would contain little or no information about most of the possible 30-day sequences.

1.3.2 The joint distribution in autoregressive form

The rain record showed how fast a distribution over a whole sequence grows. The chain rule rewrites such a distribution as a chain of next-step predictions.

That form matches how sequences are used. Words arrive one after another, and what we usually want from a model is the distribution of the next word given what has been written so far, whether we are generating text, scoring a sentence, or filling in a blank. A model of those next-step distributions is already a model of the whole sequence, with nothing lost.

It is also the form in which approximations can be made. A table over all sequences has one number for each of them and no structure to take advantage of. Once the distribution is written as a chain, every factor conditions on a history, and the history is where an assumption can be applied: keep the last few tokens and drop the rest, or summarize the history with a function that has a fixed number of parameters. Either one turns an unusable table into a model that can be estimated from a finite corpus, and both are developed in the rest of this section.

Text will be our running example, though the construction applies to any ordered data, including music, DNA, and a sequence of measurements. We write the vocabulary of possible tokens as $\mathcal{V}$ and its size as $V = |\mathcal{V}|$.

Let $W_1, W_2, \ldots, W_T$ be the random tokens in a sequence, and let $w_1, w_2, \ldots, w_T$ denote a particular sequence of values. The chain rule gives

$$p(w_1, \ldots, w_T) = p(w_1) \prod_{t=2}^{T} p(w_t \mid w_1, \ldots, w_{t-1}). \tag{1.8}$$

It is convenient to abbreviate $(w_1, \ldots, w_{t-1})$ as $w_{1:t-1}$, so the same factorization can be written as

$$p(w_{1:T}) = \prod_{t=1}^{T} p(w_t \mid w_{1:t-1}),$$

where the factor for $t = 1$ is $p(w_1)$. It says that we can describe the first token, then the second given the first, then the third given the first two, and so on. Equation (1.8) is an identity: it makes no independence assumption and discards no part of the history. For the same reason it does not by itself make the model any smaller. If every next-token distribution may depend on the entire history, the conditional tables together hold as many free parameters as the joint distribution they came from, which for the 30-day weather record is the same $2^{30} - 1$ numbers as before.

> ★ **Chain rule versus assumptions**
>
> The chain rule turns a joint distribution into a sequence of next-step distributions. We need additional assumptions before the number of parameters becomes smaller.

Definition 1.1 (Autoregressive sequence model)**.** An **autoregressive sequence model** specifies the joint distribution of a sequence through a left-to-right collection of conditional distributions, with one distribution $p(w_t \mid w_{1:t-1})$ for each next token. For a discrete vocabulary, each next-token distribution is categorical.

Thus a model that predicts only the next token is also a model of the entire sequence. The product of all of its next-token probabilities gives the joint probability of that sequence. Conversely, sampling once from each conditional distribution, from left to right, produces a sample from the joint distribution.

Sentences do not all have the same length, so it is useful to add a special end-of-sequence token EOS. If $w_{T+1} = \text{EOS}$, then

$$p(w_{1:T}, \text{EOS}) = \prod_{t=1}^{T+1} p(w_t \mid w_{1:t-1}). \tag{1.9}$$

The probability of ending is therefore treated in exactly the same way as the probability of producing any other token.[1] We will also use a special **beginning-of-sequence** token, written BOS. It is an artificial token placed before the first real token of every sequence: it tells the model that generation is starting. For example, the first prediction can be written $p(w_1 \mid \text{BOS})$. Thus possible next tokens belong to $\mathcal{V} \cup \{\text{EOS}\}$, while a context may also contain BOS.

Example 1.2 (A short sequence)**.** Suppose a model uses one-token contexts and assigns

$$p(\text{the} \mid \text{BOS}) = 0.3, \qquad p(\text{cat} \mid \text{the}) = 0.2, \qquad p(\text{EOS} \mid \text{cat}) = 0.5.$$

The probability that it produces "the cat" and then stops is

$$p(\text{the}, \text{cat}, \text{EOS} \mid \text{BOS}) = 0.3 \times 0.2 \times 0.5 = 0.03.$$

Each conditional uses exactly one preceding token. Their product is the probability of the whole sequence. △

1.3.3 Finite-context approximations

The exact factorization in (1.8) becomes difficult to use as the sequence grows. The conditional distribution at position t may depend on an arbitrarily long prefix, and a separate table row would be needed for every possible prefix. A finite-order model replaces the entire prefix by a fixed-length context.

Definition 1.3 (Order-k approximation)**.** An **order-k autoregressive sequence model**, also called a **kth-order Markov model**, uses only the preceding k tokens to predict the next token. It makes the finite-order Markov approximation

$$p(w_t \mid w_{1:t-1}) \approx p_k(w_t \mid w_{t-k:t-1}). \tag{1.10}$$

Here p denotes an unrestricted distribution over sequences, whereas p_k denotes the order-k model. Within that model, the finite-context relation is exact:

$$p_k(w_t \mid w_{1:t-1}) = p_k(w_t \mid w_{t-k:t-1}).$$

For an order-k model, we prepend k copies of BOS, so that the first few predictions also have a k-token context.

The context $w_{t-k:t-1}$ plays the role of the state. The cases $k = 0, 1, 2$ are especially helpful for understanding the assumption:

Order 0: The next token depends on no previous token: $p(w_t \mid w_{1:t-1}) \approx p_0(w_t)$. Before stopping, its next-token distribution ignores all previous tokens.

Order 1: The next token depends only on the previous token: $p(w_t \mid w_{1:t-1}) \approx p_1(w_t \mid w_{t-1})$. Once the previous token is known, the ones before it say nothing further about what comes next. This is the **Markov property**.

Order 2: The next token depends on the previous two tokens: $p(w_t \mid w_{1:t-1}) \approx p_2(w_t \mid w_{t-2}, w_{t-1})$.

We will use the same context-to-next-token distribution at every position in the sequence. In other words, the conditional probability does not depend on t, which makes the model a **time-homogeneous** Markov chain whose state is the last k tokens. Its transition table has one row for each of the V^k contexts.

Finite context makes the model manageable, but its size still grows rapidly. Ignoring boundary symbols, an order-k model has as many as V^k contexts. For every context it must specify a categorical distribution over V possible next tokens. Such a distribution has $V - 1$ free parameters: the probabilities of the V possible

[1] For variable-length generation, we assume that the procedure eventually produces EOS with probability one. Under this condition, the probabilities in (1.9) define a probability distribution over finite sequences. Otherwise, some probability mass could be assigned to sequences that never end.

next tokens must sum to one, so once $V-1$ are chosen, the last is determined. Thus a complete conditional table has as many as

$$V^k(V-1) \tag{1.11}$$

free parameters. This count omits the initial or BOS row. For the binary order-1 rain model, it gives two transition parameters, and the probability of rain on the first day makes three numbers in all. Equivalently, the table contains on the order of V^{k+1} entries. If $V = 10{,}000$, even an order-2 table has up to 10^8 contexts and on the order of 10^{12} entries. Most of these contexts will not appear in a realistically sized corpus.

The approximation also determines which relationships the model can express. For example, in

"The cats that I saw yesterday at the park _____."

the correct choice "were" depends on "cats," which is many words earlier. Any context that does not reach back to "cats" must predict without that information. Increasing k can include more such relationships, but it also increases the number of contexts that must be learned.

1.3.4 Learning conditional tables from counts

Suppose we are given a corpus $\mathcal{D}$ of token sequences. For an order-k model, let $c \in (\mathcal{V} \cup \{\text{BOS}\})^k$ denote a context of length k, and define

$$\begin{aligned} N(c,w) &= \text{the number of times token } w \text{ follows context } c \text{ in } \mathcal{D}, \\ N(c) &= \sum_{u \in \mathcal{V} \cup \{\text{EOS}\}} N(c,u). \end{aligned}$$

Here $N(c)$ is the total number of observed continuations of c. For an observed context, define the **empirical conditional distribution**, or **relative-frequency estimate**, by

$$\hat{p}_k(w \mid c) = \frac{N(c,w)}{N(c)}. \tag{1.12}$$

This estimate is defined only for an observed context with $N(c) > 0$, and it assigns probability zero to a continuation with $N(c,w) = 0$.[2] The entries in every observed row form a probability distribution because

$$\sum_{w \in \mathcal{V} \cup \{\text{EOS}\}} \hat{p}_k(w \mid c) = \frac{\sum_{w \in \mathcal{V} \cup \{\text{EOS}\}} N(c,w)}{N(c)} = 1.$$

This is a count-based estimate constructed from the corpus. The next chapter will show that it is also the maximum-likelihood estimate for a categorical next-token distribution.

Example 1.4 (An order-two count table)**.** Consider a corpus consisting of the following four sentences:

the cat sat,	the cat slept,
the dog sat,	a cat sat.

For an order-2 model, pad each sentence with two BOS tokens and add one EOS token. For example, the first sequence becomes

$$(\text{BOS}, \text{BOS}, \text{the}, \text{cat}, \text{sat}, \text{EOS}).$$

Counting each length-two context and its following token gives

[2] Practical count-based sequence models use additional methods to handle unseen contexts and continuations. Smoothing and backoff are not covered here.

Context c	Nonzero counts $N(c, w)$	Estimated distribution $\hat{p}_2(w \mid c)$
(BOS, BOS)	the: 3, a: 1	the: 3/4, a: 1/4
(BOS, the)	cat: 2, dog: 1	cat: 2/3, dog: 1/3
(BOS, a)	cat: 1	cat: 1
(the, cat)	sat: 1, slept: 1	sat: 1/2, slept: 1/2
(the, dog)	sat: 1	sat: 1
(a, cat)	sat: 1	sat: 1
(cat, sat)	EOS: 2	EOS: 1
(cat, slept)	EOS: 1	EOS: 1
(dog, sat)	EOS: 1	EOS: 1

For instance, the context (the, cat) occurs twice. It is followed once by "sat" and once by "slept," so

$$\hat{p}_2(\text{sat} \mid \text{the}, \text{cat}) = \hat{p}_2(\text{slept} \mid \text{the}, \text{cat}) = \frac{1}{2}.$$

This entire model has been learned by counting and normalizing each context row. △

For each context, count its observed continuations and divide by the row total. The result is a categorical next-token distribution for that context.

1.3.5 Generating a sequence one token at a time

Once the conditional table is available, generation is a repeated categorical sampling procedure. For an order-k model:

1. Initialize the context with k copies of BOS.
2. Sample the next token w from $\hat{p}_k(\cdot \mid c)$.
3. If $w = \text{EOS}$, stop. Otherwise append w to the sequence.
4. Form the new context by discarding the oldest context token and appending w, and then return to step 2. For $k = 0$, the context remains empty.

Each draw can be carried out by the categorical inverse-CDF procedure from the probability review: divide $[0, 1)$ into intervals whose lengths are the row probabilities and use a uniform draw to select an interval.

Example 1.5 (Sampling from the table)**.** Use the order-2 model in Example 1.4. Begin with (BOS, BOS) and suppose the successive draws select "the," then "cat," then "slept." The contexts evolve as follows:

$$\begin{aligned} (\text{BOS}, \text{BOS}) &\longrightarrow \text{the}, \\ (\text{BOS}, \text{the}) &\longrightarrow \text{cat}, \\ (\text{the}, \text{cat}) &\longrightarrow \text{slept}, \\ (\text{cat}, \text{slept}) &\longrightarrow \text{EOS}. \end{aligned}$$

Conditioned on the initial BOS context, the probability of this particular generated sequence is the product of the selected probabilities:

$$\frac{3}{4} \times \frac{2}{3} \times \frac{1}{2} \times 1 = \frac{1}{4}.$$

Multiplying these conditional probabilities gives the probability of the whole generated sequence. This is the chain rule applied to the finite-context model. △

Sampling is different from always selecting the most probable next token. Always taking the largest-probability continuation produces a single deterministic path (apart from ties). Sampling preserves the alternatives and their probabilities, so repeated runs can produce different sequences.

Exercise 1.6 (Hand-sampling an order-two model)**.** Use the table in Example 1.4.

1. Starting from (BOS, BOS), suppose the first draw selects "a." Complete the generated sequence and find its probability.
2. Find the probability of generating "the dog sat" and then stopping.
3. Explain why the order-2 model cannot generate "a cat slept," even though every individual word occurs in the corpus.

Solution. After "a," every relevant row is deterministic:

$$(\text{BOS}, \text{a}) \to \text{cat}, \qquad (\text{a}, \text{cat}) \to \text{sat}, \qquad (\text{cat}, \text{sat}) \to \text{EOS}.$$

Thus the generated sentence is "a cat sat," with probability $\frac{1}{4} \times 1 \times 1 \times 1 = \frac{1}{4}$.

For "the dog sat," the probability is

$$\hat{p}_2(\text{the} \mid \text{BOS}, \text{BOS})\hat{p}_2(\text{dog} \mid \text{BOS}, \text{the})\hat{p}_2(\text{sat} \mid \text{the}, \text{dog})\hat{p}_2(\text{EOS} \mid \text{dog}, \text{sat}) = \frac{3}{4} \times \frac{1}{3} \times 1 \times 1 = \frac{1}{4}.$$

Finally, the only observed continuation of (a, cat) is "sat," so $\hat{p}_2(\text{slept} \mid \text{a}, \text{cat}) = 0$. Therefore the model assigns zero probability to "a cat slept."

△

1.3.6 Expressiveness versus data

The order k controls both what the model can represent and how much data it needs. These two effects pull in opposite directions.

If k is small, many distinct histories are mapped to the same context. This allows the model to pool counts and estimate each row from more observations, but it also forces histories with genuinely different next-token behavior to share a distribution. An order-0 word model, for example, can learn that "the" is common but cannot learn that "the cat" is more natural than "the the." A small-context model therefore tends to **underfit**: it ignores useful structure that is present in the sequence.

If k is large, the model can distinguish more histories. It is more expressive, but the V^k possible contexts divide a fixed corpus into many small groups. Most contexts may occur once or not at all. A context seen once has a row that assigns probability one to its single observed continuation, while an unseen context has no estimated row. As the context approaches the length of a training sequence, generation increasingly reproduces pieces of the corpus instead of combining evidence across related contexts. This is memorization caused by sparse data, not proof that the sequence is deterministic.

The small corpus above makes the distinction concrete. An order-1 model pools all occurrences of "cat," regardless of what precedes it. Since "cat" is followed twice by "sat" and once by "slept," it assigns

$$\hat{p}_1(\text{sat} \mid \text{cat}) = \frac{2}{3}, \qquad \hat{p}_1(\text{slept} \mid \text{cat}) = \frac{1}{3}.$$

It can therefore generate the new combination "a cat slept." The order-2 model makes the more specific distinction between (the, cat) and (a, cat); on this small dataset, that extra expressiveness also prevents it from making the new combination. Neither behavior is always correct. We need enough context to express important dependencies and enough observations to estimate the resulting conditional distributions.

Context size	Benefit	Cost with fixed data
Small k	More observations per context	Important history is ignored
Large k	More distinctions between histories	Sparse and unseen contexts

★ **Context must match data**

A larger context lets the model use more information from the preceding tokens. However, longer contexts occur less often in a fixed corpus, so their conditional probabilities may be estimated from very few observations. We must balance a model's ability to represent detailed patterns against the amount of data available to estimate those patterns.

1.3.7 Beyond a separate table for every context

The order-1 and order-2 examples leave us with an uncomfortable choice. The order-1 model combines many histories into the same short context, so it has more observations for each row but forgets useful information. The order-2 model remembers more, but learns the rows for (the, cat) and (a, cat) separately. What if we want to remember a richer context without treating every context as a completely separate case?

The count table cannot give us that. It keeps one row for every context, so its size is not something we choose: it is whatever the vocabulary and the context length make it. With $V = 10{,}000$, the count (1.11) is about 10^{12} free parameters at $k = 2$ and about 10^{24} at $k = 5$. A model whose number of parameters grows in this way, with the number of distinct contexts rather than being fixed in advance, is called **non-parametric**.

Storage is the smaller of the two problems. Each row is estimated from the occurrences of its own context and from nothing else, while a corpus of 10^9 tokens contains at most 10^9 contexts however many rows the table has. At $k = 5$, all but a vanishing fraction of the rows are therefore empty, and an empty row means the model has learned nothing whatsoever about what follows that context. Past a modest k, the table can no longer be learned at all, whatever the storage budget.

Instead of looking up a row in a table, we can use one function for all contexts. We give it a context c, and it returns the probabilities of the possible next tokens:

$$c \longmapsto f_\theta(c) = \boldsymbol{\pi}_\theta(c), \qquad p_\theta(W_t = w \mid c) = \pi_{\theta,w}(c). \tag{1.13}$$

Here θ collects all the adjustable numbers in the function, and its length is chosen when the model is designed. A **parametric model** is a model controlled by such a fixed set of parameters. The important difference is how the parameters are used: a lookup table spends a separate row on every context, whereas f_θ answers every context with the same θ. As a result, what the model learns from one context can affect its predictions in another.

Example 1.7 (Next-step distributions)**.** The output of the function depends on the quantity we want to generate. For rain or no rain, it could return one number $q_\theta(c)$, interpreted as the probability of rain. For a token, it returns a list of probabilities $\boldsymbol{\pi}_\theta(c)$, one for each possible next token. In both cases the function produces a probability distribution for the next observation, rather than simply guessing one value. △

Two ideas make such a function possible. The first is to stop treating tokens as unrelated symbols. In a count table, the rows for (the, cat) and $(\text{a}, \text{kitten})$ have nothing to do with each other, because two symbols are either identical or unrelated and counts collected for one row never inform another. A parametric model instead represents each token by a vector of real numbers, called an **embedding**, so that a context becomes a collection of vectors rather than a collection of symbols. Vectors admit degrees of similarity: "cat" and "kitten" can sit close together, and a function that varies smoothly with its input then returns similar next-token distributions for the two contexts above. The embeddings are themselves part of θ and are learned along with everything else, so the model decides which tokens belong near each other.

The second idea is to build f_θ by composing many simple steps, each a linear map of its input followed by a fixed nonlinear function applied to every coordinate. This is a **neural network**, and we do not need its internal details here. The nonlinearity is what makes the composition worth anything, since composing linear maps only produces another linear map; with it, a moderate number of parameters describes a rich family of functions of the context vectors. The size of θ is set by the widths of those maps and not by the

number of contexts, so the context can be made longer without the parameter count following V^k. That is how a language model conditions on thousands of previous tokens:

Model	Context	Free parameters
Order-2 count table	2 tokens	about 10^{12}
Order-5 count table	5 tokens	about 10^{24}
Neural language model	thousands of tokens	10^9 to 10^{12}

The two tables assume $V = 10{,}000$, and the last row is the range spanned by the language models in use today. The neural model reaches back hundreds of times further than the order-5 table while holding far fewer parameters, because its parameters are shared across contexts rather than divided among them.

Changing how we obtain the next-token probabilities does not change the autoregressive model itself. We still have

$$p_\theta(w_{1:T}) = \prod_{t=1}^{T} p_\theta(w_t \mid w_{1:t-1}), \tag{1.14}$$

and generation still means obtaining a distribution, sampling a token, appending it to the context, and repeating.

One question remains: how should we choose θ? For the table, we counted the observed continuations and normalized each row. For a more general function, there may be no such direct formula. The next chapter develops the general answer: choose parameters that make the observed data probable. The connection to this section comes from

$$\log p_\theta(w_{1:T}) = \sum_{t=1}^{T} \log p_\theta(w_t \mid w_{1:t-1}).$$

The probability of a sequence is a product of next-token probabilities, so its log probability is a sum of next-token terms. We will make use of this fact when we study how to learn the parameters.

★ **What stays the same**

A table and a shared function give us different ways to obtain the next-token probabilities. The chain-rule factorization and the step-by-step generation procedure stay the same.

1.4 Information theory and machine learning

Information theory deals with quantifying information and the rules that govern its transmission, storage, and transformation from one form to another. It has applications in communications, data storage, machine learning, and biology. In machine learning it can be used to help better understand relationships between knowns and unknowns, design loss functions, and establish fundamental limits on how well we can do with a certain amount of data (regardless of the type of algorithm and computational resources).

1.4.1 Quantifying uncertainty

Let X be a Bernoulli random variable that is equal to 1 if it is raining in Seattle and 0 otherwise. Similarly, let Y indicate whether it's raining in Phoenix. How much information do X and Y provide us? Alternatively, before they are revealed, how uncertain are we about X and about Y? Can we measure the information content of a random variable, or equivalently, our uncertainty about them.

Let's look at specific outcomes for each variable:

Raining in Seattle ($X = 1$): This carries a fair amount of information, since rain in Seattle is almost a 50/50 proposition.

Not raining in Phoenix ($Y = 0$): This carries little information, since the outcome is expected and has high probability.

Raining in Phoenix ($Y = 1$): This carries a lot of information, since the outcome is unlikely and surprising.

So as a function of probability, the amount of information of a given statement decreases as the probability increases. If the probability of an outcome is p, what is a good function describing the amount of information we gain from learning that the outcome has occurred? A good choice is the *self-information* function $I(p) = \log \frac{1}{p}$, shown in Figure 1.1 when the base of the log is 2. Then the information content of the statement '$X = x_i$' is

$$I(p(x_i)) = \log \frac{1}{p(x_i)}.$$

And the amount of information *on average* for a random variable X that takes values in the set $\mathcal{X} = \{x_1, \ldots, x_m\}$ is the **entropy**,

$$H(X) = \mathbb{E}\left[\log \frac{1}{p(X)}\right] = \sum_{i=1}^{m} p(x_i) \log \frac{1}{p(x_i)}, \tag{1.15}$$

where for continuous RVs, the sum must be replaced with an integral. If the log is base 2, then the unit is a *bit*.

If there are m different possible outcomes, then

$$0 \leq H(X) \leq \log m,$$

with $H(X) = 0$ when the outcome is certain and $H(X) = \log m$ when all outcomes are equally likely. For example, a fair die has entropy $\log_2 6 \simeq 2.58$ bits, the maximum possible for six outcomes. †

Example 1.8 (Bounds on entropy)**.** Show that $0 \leq H(X) \leq \log m$ for a random variable with at most m possible values, and identify the distributions that attain each bound.

Solution. Write $p_i = p(x_i)$, with the convention that a term with $p_i = 0$ contributes nothing to either sum below.

Lower bound. Since $0 \leq p_i \leq 1$, each term $p_i \log \frac{1}{p_i}$ is non-negative, so $H(X) \geq 0$. The sum vanishes only if every term does, which happens only when each p_i is 0 or 1, that is, when X is deterministic.

Upper bound. Let u be the uniform distribution on $\mathcal{X}$, so that $u(x_i) = 1/m$. Then

$$\log m - H(X) = \sum_{i=1}^{m} p_i \log \frac{p_i}{u(x_i)} = D(p \,\|\, u) \geq 0,$$

where the last step is the non-negativity of relative entropy, (1.17) below, which holds with equality if and only if the two distributions coincide (Exercise 1.17). So $H(X) \leq \log m$, with equality exactly when $p = u$, that is, when X is uniform.

△

Exercise 1.9 (Binomial versus uniform)**.** Which has higher entropy: $X \sim \mathrm{Bin}(2, 1/2)$ or Y uniform on $\{0, 1, 2\}$? Compute both. △

Example 1.10 (Entropy of an English letter)**.** A letter is drawn at random from English text, over the 26 letters A–Z. How much entropy does it carry if every letter is equally likely? How much does it carry under the actual letter frequencies of English, listed in Table 1.1?

Solution. Equally likely letters give the largest possible value, $\log_2 26 \simeq 4.70$ bits. The real frequencies are far from uniform: E is more than a hundred times as common as Z. Weighting by actual probability, gives

4.18 bits. Models that also use the dependence of each letter on the ones before it give lower values still: Shannon estimated the entropy of English at around 1 bit per character. Language models exploit exactly this predictability.

△

Example 1.11 (Entropy of a Bernoulli variable)**.** Let $X \sim \text{Ber}(p)$, so that $X = 1$ with probability p and $X = 0$ with probability $1 - p$. What is $H(X)$?

Solution. The sum in (1.15) has only two terms, so

$$H(X) = p \log \frac{1}{p} + (1-p) \log \frac{1}{1-p},$$

which depends on the distribution only through p.

△

This special case is common enough to have its own name and notation: the **binary entropy function**

$$H_b(p) = p \log \frac{1}{p} + (1-p) \log \frac{1}{1-p}$$

is the entropy of any experiment with two outcomes of probabilities p and $1 - p$. For example,

$$\begin{aligned}
H(\text{Fair coin}) &= H_b\left(\frac{1}{2}\right) = 1, \\
H(\text{6 on a die}) &= H_b\left(\frac{1}{6}\right) = 0.65, \\
H(\text{Rain/no rain in Seattle}) &= H_b\left(\frac{150}{365}\right) = 0.977, \\
H(\text{Rain/no rain in Phoenix}) &= H_b\left(\frac{33}{365}\right) = 0.43784, \\
H(\text{Rain/no rain in the Sahara}) &= H_b\left(\frac{1}{365}\right) = 0.027267.
\end{aligned}$$

The plot for binary entropy is given in Figure 1.1. The maximum entropy is 1 bit. This makes sense since we can represent the outcome with 1 bit. Random variables with equal chances of 0 and 1 have the highest entropy (and maximum uncertainty). Those with predictable outcomes have lower entropies.

Example 1.12 (Winning the lottery)**.** In a lottery, the jackpot is won by matching five distinct numbers chosen from 70 plus a bonus number chosen from 25. How much information does a ticket holder gain from each of the two outcomes, and what is the entropy of the win/lose variable?

Solution. A ticket wins with probability $1/\left(\binom{70}{5} \cdot 25\right) = 1/302{,}575{,}350$. Learning that a ticket won conveys $\log_2 302{,}575{,}350 \simeq 28.2$ bits; learning that it lost conveys $\simeq 4.8 \times 10^{-9}$ bits (you were almost sure this will happen). The entropy of the win/lose variable is accordingly minute, $H_b\big(1/302{,}575{,}350\big) \simeq 10^{-7}$ bits. This example shows that an outcome can be extremely surprising even when the variable carries almost no information on average.

△

Entropy was introduced by Shannon in his article "A mathematical theory of communication" in 1948, where he also popularized the term *bit* (binary digit).

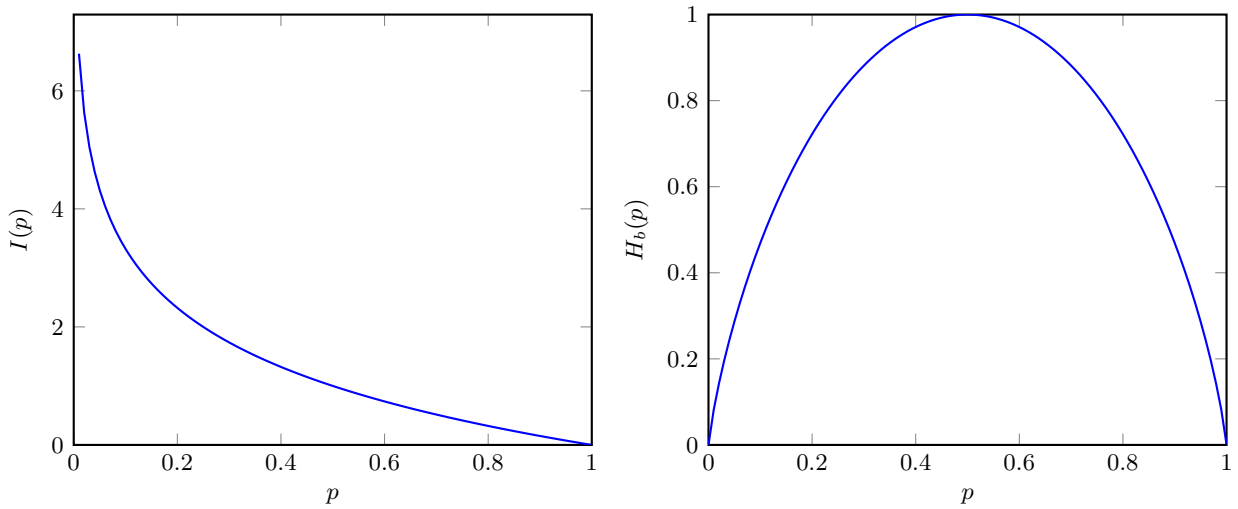


Figure 1.1: Self-information (left) for an event with probability p and binary entropy (right) for a Bernoulli RV with probability of success equal to p.

> "My greatest concern was what to call it. I thought of calling it 'information,' but the word was overly used, so I decided to call it 'uncertainty.' When I discussed it with John von Neumann, he had a better idea. Von Neumann told me, 'You should call it entropy, for two reasons. In the first place your uncertainty function has been used in statistical mechanics under that name, so it already has a name. In the second place, and more important, no one really knows what entropy really is, so in a debate you will always have the advantage." – Claude Shannon, Scientific American (1971), volume 225, page 180.

1.4.2 Data compression and the meaning of entropy

Entropy measures average uncertainty, but it also has a direct operational meaning: it provides a fundamental bound on how compactly we can represent information. Specifically, entropy bounds the performance of **lossless data compression**: no data compression method, regardless of computational complexity, can beat entropy.

Suppose we want to represent the outcomes of a random variable as binary strings. That is we want to construct a **source code**, consisting of **codewords**, where each codeword represents an outcome. We aim for a code that is as short as possible on average. The strategy is intuitive: assign short codewords to likely outcomes and long codewords to unlikely ones. The question is how well we can do.

Example 1.13 (Coding the alphabet)**.** English text is to be transmitted one letter at a time, over the 26 letters A–Z.

(a) Design a code that spends the same number of binary digits on every letter. How many digits does each letter need, and how many does the word "TEA" take?

(b) Letters are not equally common: E and T occur far more often than Q and Z. Design a code that exploits this, and encode "TEA" with it.

Solution. (a) Four binary digits distinguish only $2^4 = 16$ letters, so each letter needs $\lceil \log_2 26 \rceil = 5$ digits, and any assignment of the 26 letters to distinct 5-digit strings will do. Every letter costs the same 5 bits, so "TEA" takes $3 \times 5 = 15$ bits.

(b) Give the frequent letters the short codewords and the rare ones the long codewords. The best such code is produced by Huffman's algorithm. (But any code that follows the guideline of short codewords for frequent outcomes is fine for this exaomple.) Table 1.1 gives the code it builds from the English letter frequencies.

Table 1.1: A Huffman code for the 26 letters, built from the English letter frequencies (%) shown beside each letter. Frequent letters get the short codewords.

	Codeword	Len	%		Codeword	Len	%		Codeword	Len	%
E	`100`	3	12.7	D	`11111`	5	4.3	P	`110001`	6	1.9
T	`000`	3	9.1	L	`11110`	5	4.0	B	`110000`	6	1.5
A	`1110`	4	8.2	C	`01000`	5	2.8	V	`001110`	6	1.0
O	`1101`	4	7.5	U	`01001`	5	2.8	K	`0011111`	7	0.8
I	`1011`	4	7.0	M	`00101`	5	2.4	X	`00111100`	8	0.2
N	`1010`	4	6.7	W	`00110`	5	2.4	J	`001111010`	9	0.2
S	`0111`	4	6.3	F	`00100`	5	2.2	Q	`0011110110`	10	0.1
H	`0110`	4	6.1	G	`110010`	6	2.0	Z	`0011110111`	10	0.1
R	`0101`	4	6.0	Y	`110011`	6	2.0				

With this code "TEA" is `000 100 1110`, that is 10 bits in place of the 15 of part (a). Averaged over the letter frequencies, the code spends 4.21 bits per letter rather than 5. Samuel Morse arrived at the same principle by hand in the 1830s. His telegraph code spells E as a single dot and saves the long sequences for rare letters such as Q and Z. However, his code is over three symbols: dot, dash, pause.

That 4.21 is already close to the entropy of the letter distribution, i.e., 4.18 bits. The rest of this subsection explains why that is no coincidence, and why no code can average less.

△

It turns out that the ideal code length for an outcome with probability p is $\log_2(1/p)$ bits, exactly the self-information introduced earlier. A likely outcome (p close to 1) needs a short codeword; a surprising outcome (p close to 0) can have a long one. Averaging over outcomes, the minimum achievable average code length is the entropy $H(X)$. This is **Shannon's source coding theorem**.

> ★ **Shannon's source coding theorem**
>
> If we know the distribution of a source, we can compress close to the entropy limit by assigning roughly $\log_2(1/p)$ bits to an outcome with probability p: short codewords for likely outcomes, long ones for unlikely outcomes. No lossless code averages fewer than $H(X)$ bits per symbol, so nobody can do better.

Exercise 1.14 (Optimal binary code)**.** A source emits four symbols A, B, C, D with probabilities 1/2, 1/4, 1/8, 1/8.

(a) Compute the entropy $H(X)$.

(b) Consider the binary code A $\to$ `0`, B $\to$ `10`, C $\to$ `110`, D $\to$ `111`. Verify that the average code length equals $H(X)$.

(c) Verify that each codeword length equals $\log_2(1/p)$ for the corresponding probability.

△

1.4.3 Relative entropy, or what if we get the distribution wrong?

The previous subsection showed that knowing the true distribution p lets us compress down to $H(X)$ bits per symbol. But what if we do not know p and instead design our code for a different distribution q?

Let X be a random variable with possible values $\mathcal{X}$ and distribution p, and let q be another distribution over $\mathcal{X}$. A code optimized for q assigns roughly $\log(1/q(x))$ bits to outcome x. When the data actually follow p,

the average code length becomes the **cross-entropy**

$$H(p, q) = \sum_{x \in \mathcal{X}} p(x) \log \frac{1}{q(x)}. \tag{1.16}$$

Example 1.15 (A code for DNA)**.** A DNA sequence is written over $\mathcal{X} = \{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}\}$, and the four bases occur with probabilities

$$p(\mathrm{A}) = \tfrac{1}{2}, \qquad p(\mathrm{C}) = \tfrac{1}{4}, \qquad p(\mathrm{G}) = p(\mathrm{T}) = \tfrac{1}{8}.$$

Not knowing this, we design our code for the uniform distribution q instead. How many bits per base does that code use, and how many would the best code use?

Solution. The best achievable average is the entropy of p,

$$H(X) = \tfrac{1}{2} \log_2 2 + \tfrac{1}{4} \log_2 4 + 2 \cdot \tfrac{1}{8} \log_2 8 = 1.75 \text{ bits per base.}$$

A code built for the uniform q spends $\log_2 4 = 2$ bits on every base regardless of which one it is, so by (1.16) its average length under the real data is

$$H(p, q) = \sum_{x \in \mathcal{X}} p(x) \log_2 4 = 2 \text{ bits per base.}$$

Ignoring the composition of the sequence therefore costs a quarter of a bit on every base.

△

Since the best possible average length is $H(X)$, the penalty for using the wrong distribution is

$$H(p, q) - H(X) = D(p \| q),$$

the *relative entropy*, or *Kullback–Leibler (KL) divergence*:

$$D(p \| q) = \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)}. \tag{1.17}$$

> ★ **Penalty for the wrong distribution**
> The relative entropy $D(p\|q)$ is exactly how many extra bits per symbol we waste by compressing with q instead of the true distribution p. The larger this value, the worse q approximates p.

Example 1.16 (A code for DNA, continued)**.** Take p and q from Example 1.15. Compute $D(p\|q)$ directly from (1.17), and check it against the quarter of a bit that example found wasted.

Solution. Evaluating (1.17) with q uniform,

$$D(p \| q) = \tfrac{1}{2} \log_2 \frac{1/2}{1/4} + \tfrac{1}{4} \log_2 \frac{1/4}{1/4} + 2 \cdot \tfrac{1}{8} \log_2 \frac{1/8}{1/4} = \tfrac{1}{2} - \tfrac{1}{4} = 0.25 \text{ bits,}$$

which is $H(p, q) - H(X) = 2 - 1.75$, the quarter of a bit expected. The term for A is positive and the terms for G and T are negative, but A is the common base and those two are rare, so the balance is a quarter of a bit lost.

△

The compression picture makes the key properties of relative entropy intuitive.

Non-negativity. No code can beat the entropy bound $H(X)$, so compressing with the wrong distribution q can never be *cheaper* than using the true p. Therefore $D(p\|q) = H(p,q) - H(X) \geq 0$, with equality if and only if $q = p$. †

Exercise 1.17 (Non-negativity of relative entropy)**.** Prove that $D(p\|q) \geq 0$ with equality if and only if $p = q$. *Hint:* use $\ln t \geq 1 - \frac{1}{t}$ for $t > 0$, with equality iff $t = 1$, applied to $t = p(x)/q(x)$. △

When relative entropy is large or infinite. If q assigns a tiny probability to an outcome that p considers common, the code designed for q wastes a huge number of bits on that outcome: $\log(1/q(x))$ is large when $q(x)$ is small. In the extreme case where $q(x) = 0$ for some x with $p(x) > 0$, the code for q has no codeword at all for an outcome that actually occurs; no finite number of bits suffices, and $D(p\|q) = \infty$. In words, if the approximate distribution considers an event impossible but the true distribution does not, no amount of coding can compensate for that mistake.

Example 1.18 (Infinite relative entropy)**.** Keep the DNA distribution p of Example 1.15, and let q assign 1/2 to A, 1/4 to C, 1/4 to G, and 0 to T. What is $D(p\|q)$?

Solution. It is infinite. Every base but T contributes a finite term, while T contributes $\frac{1}{8}\log\frac{1/8}{0} = \infty$, since $p(\mathrm{T}) = 1/8 > 0$ and $q(\mathrm{T}) = 0$. In coding terms, a code designed for q has no codeword at all for T, so a sequence containing one cannot be sent at any rate.

△

Asymmetry: choosing the direction. Relative entropy is not symmetric: $D(p\|q) \neq D(q\|p)$ in general.

Example 1.19 (Fair versus biased coin)**.** Let p be the distribution of a fair coin and q that of a coin with $\Pr(\text{heads}) = 0.9$. Compute both directions and compare them.

Solution. In bits,

$$D(p\|q) = \frac{1}{2}\log_2\frac{1/2}{0.9} + \frac{1}{2}\log_2\frac{1/2}{0.1} \simeq 0.74,$$
$$D(q\|p) = 0.9\log_2\frac{0.9}{1/2} + 0.1\log_2\frac{0.1}{1/2} \simeq 0.53,$$

so the two differ: relative entropy is not symmetric.

△

1.4.3.1 Relative entropy as a cost function

So far $D(p\|q)$ has measured the price of compressing with the wrong distribution. The same number measures how good an approximation q is to p: it is zero exactly when the two agree, it grows as they separate, and it is infinite when q rules out an outcome that p allows. That makes it a natural quantity to minimize when fitting a model q to data drawn from p, which is how it usually reaches machine learning.

The two directions also answer different compression questions. In $D(p\|q)$, we compress data from the true distribution p using a code designed for our estimate q; this is the "forward" direction used above. In $D(q\|p)$, we compress data from our model q using a code designed for the truth p. When does the reverse direction matter?

Suppose we are fitting an approximate distribution q to a true distribution p. Minimizing $D(p\|q)$ forces q to place probability mass wherever p does (otherwise the penalty is infinite), so q tends to cover all of p's support, sometimes spreading mass too broadly. Minimizing $D(q\|p)$ instead penalizes q for placing mass where p does not: if $q(x) > 0$ but $p(x) = 0$, then $\log(p(x)/q(x))$ contributes $-\infty$, which is never chosen. So q concentrates on a high-probability region of p, potentially ignoring some modes. Both directions arise

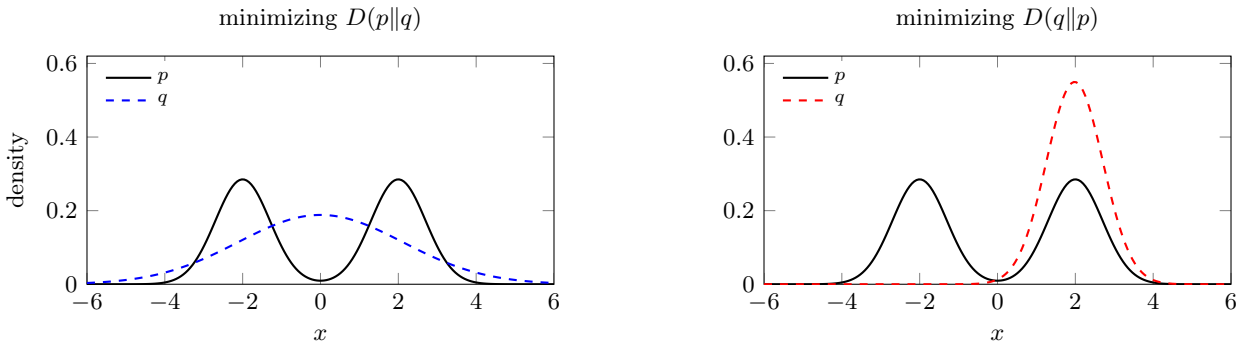


Figure 1.2: Fitting a single Gaussian q to the two-mode density p of Example 1.20. Minimizing $D(p\|q)$ (left) spreads q over both modes and peaks where p is near zero. Minimizing $D(q\|p)$ (right) puts q on one mode and gives up the other.

in machine learning: $D(p\|q)$ as the training objective for generative models via maximum likelihood, and $D(q\|p)$ as the objective in variational inference, where q is a tractable approximation to an intractable posterior p.

Example 1.20 (One Gaussian for two)**.** Let p be an equal mixture of two Gaussians,

$$p = \tfrac{1}{2}\,\mathcal{N}(-2, 0.7^2) + \tfrac{1}{2}\,\mathcal{N}(2, 0.7^2),$$

and let the model q be a single Gaussian $\mathcal{N}(\mu, \sigma^2)$, which cannot reproduce two modes. Which q minimizes $D(p\|q)$, and which minimizes $D(q\|p)$?

Solution. For a Gaussian model, minimizing $D(p\|q)$ matches the mean and the variance of p. Here p has mean 0 and variance $0.7^2 + 2^2 = 4.49$, so

$$\mu = 0, \qquad \sigma = 2.12, \qquad D(p\|q) = 0.61 \text{ bits}.$$

This q stretches across both modes and puts its own peak at $x = 0$, where p has almost no mass at all.

Minimizing $D(q\|p)$ has no closed form, but a numerical search gives

$$\mu = 1.98, \qquad \sigma = 0.73, \qquad D(q\|p) = 0.99 \text{ bits},$$

a narrow Gaussian sitting on one mode; $\mu = -1.98$ is equivalent by symmetry. Centring at 0 is also a local minimum, with $\sigma = 1.85$ and $D(q\|p) = 1.05$ bits, but a worse one.

The two answers differ because of which distribution the average is taken under. $D(p\|q)$ averages under p, so it is charged wherever p has mass and q does not: q has to cover everything p does. $D(q\|p)$ averages under q, so q is charged only where it places mass itself, and abandoning a mode costs nothing. Figure 1.2 shows the two fits. Both behaviours, and the wider family of divergences that interpolates between them, are analysed in [1, Fig. 1].

△

Since $D(p\|q) = H(p, q) - H(X)$, for a fixed p, minimizing cross-entropy over q is the same as minimizing the relative entropy, the form in which these measures typically appear as loss functions in machine learning. In classification with one-hot labels, minimizing empirical cross-entropy is equivalent to maximizing the conditional log-likelihood.

Exercise 1.21 (Cross-entropy minimization)**.** Let $p = \text{Ber}(3/4)$ and $q = \text{Ber}(q_0)$. Write $H(p, q)$ as a function of q_0 and verify that it is minimized at $q_0 = 3/4$. △

1.4.4 Conditional Entropy and Mutual Information

We can also measure the information in multiple random variables using entropy. The information in both X and Y is denoted $H(X,Y)$ and is defined as

$$H(X,Y) = \mathbb{E}\left[\log \frac{1}{p(X,Y)}\right] = \sum_{x\in\mathcal{X}}\sum_{y\in\mathcal{Y}} p(x,y)\log\frac{1}{p(x,y)}.$$

If we know Y, how much information is left in X? This is denoted $H(X|Y)$. If, for example $X = Y + 2$, then $H(X|Y) = 0$ since if we know Y, we also know X. Conditional entropy is defined as

$$H(X|Y) = \sum_{y\in\mathcal{Y}} p(y)H(X|Y=y) = \mathbb{E}\left[\log\frac{1}{p(X|Y)}\right] = H(X,Y) - H(Y)$$

Mutual information, $I(X;Y)$, represents the amount of information that one random variable has about the other, and is defined as

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X).$$

Equivalently, it can be written as a divergence:

$$I(X;Y) = D(p_{X,Y} \,\|\, p_X p_Y).$$

While this quick overview is sufficient for our purposes in this course, if you are interested, you can check out the slides for this Short Lecture on Information Theory, or the course Mathematics of Information.